Capstone Design Project Report

Project Title: Testing Strategy and Scenario Variation

Team 25

Team Name: AutoDrive Controls

Team Members:

Dylan Vallen

Walker Sheidow

Stanley Mattam

Khalid Azzabin

Austin Decot

Electrical and Computer Engineering Department

College of Engineering

Ohio State University

ECE 4905 (10511)

Date: 17 April 2026

Document Change Notice

Date	Change
12/12/2025	Critical Design Report Released for Review
4/6/2026	Chapter 3.2 Testing Added
4/27/2026	Final Design Report Released for Review

Executive Summary

The Senior Design Capstone Program at The Ohio State University, in partnership with the Buckeye AutoDrive team, has developed a virtual simulation toolbox to address the critical need for robust autonomous vehicle software validation. Historically, the inability to simulate diverse road scenarios and generate quantitative performance metrics has hindered the software development lifecycle of the team. By leveraging tools such as RoadRunner and MATLAB, the team created a virtual proving ground that allows for the rigorous testing of autonomous behaviors without the safety risks or financial costs associated with physical track testing. This system serves as a bridge between theoretical control algorithms and real-world applications, ensuring that software is thoroughly verified before deployment.

This project includes a comprehensive library of diverse simulation scenarios, ranging from four-way intersections to pedestrian interactions, and a framework of metrics to evaluate vehicle performance. A primary innovation of this work is the Random Obstacle Generator, which programmatically injects unpredictable obstacle location into simulations. This allows the team to identify "edge case" scenarios that static testing might overlook. The system is managed through two Graphical User Interfaces (GUIs) that enable users to configure simulations, automate background execution, and receive immediate visual feedback on performance through pass/fail indicators.

Final testing and validation confirmed that the simulation system successfully records and analyzes data relative to predefined safety tolerance thresholds. By providing a repeatable and data-driven environment for software validation, this project significantly enhances the reliability of the Buckeye AutoDrive vehicle software. This development contributes to the team's success in the AutoDrive Challenge II competition.

Table of Contents

List of Figures

List of Tables

Chapter 1: Problem Definition and Preliminary Design

1.1 Problem Statement

The Buckeye AutoDrive team at The Ohio State University consisting of undergraduate and graduate students compete in the AutoDrive Challenge™ II Competition. Their team takes on the challenge of developing an autonomous vehicle that can navigate urban traffic patterns. The team takes on various tasks that are divided into sub-teams. The Buckeye AutoDrive Controls and Planning sub-team is responsible for developing and validating their autonomous driving vehicle's behavior through various urban traffic patterns.

A current problem faced by the Buckeye AutoDrive Controls team is the inability to simulate diverse test scenarios and performance metrics to validate completion rate, perception, planning, trajectory smoothness, and recovery time of the autonomous driving system. [1] The significance of their inability to test through simulation does not allow them to create a safe, repeatable, and cost-effective evaluation of software performance. Also, simulation testing allows for the collection of testing and validating different vehicle behaviors such as perception, route planning, and lane keeping. These metrics provide fast feedback to test edge cases and create a regression test library.

The capstone team has been tasked with addressing the lack of various simulated scenarios and performance evaluation tools for autonomous vehicle testing. The scope of work includes designing a set of simulation scenes to test and create key performance metrics to report from simulation data. This allows for more data-driven validation of autonomous driving behaviors in controlled environments.

1.2 Stakeholders and End Users

The groups and individuals directly impacted by the success of this project are the capstone team, the Buckeye AutoDrive Controls & Planning sub-team, and the entire Buckeye AutoDrive project team. Additional direct stakeholders include industry sponsors of the Buckeye AutoDrive team, primarily General Motors (GM) and the Society of Automotive Engineers (SAE). Additionally, The Ohio State University holds a stake, particularly the capstone director, Dr. Saideh Zia, and the faculty advisor, Dr. Lisa Fiorentini.

The Buckeye AutoDrive project team is the most directly impacted group since they are relying on project success for 20+ points of their total team score in competition. They desire a library of diverse simulated driving scenarios to test their self-driving software in various traffic patterns and climate conditions to minimize reliance on in-car testing.

Additionally, sponsors have a need for project success as their partnership builds and protects their reputations. Additionally, for many, the technology and personnel developed through this project and the AutoDrive team will benefit their direct workforce needs. With a successful project, students will develop a variety of skills as engineers that they can take to their respective companies and will reflect highly on the teams, sponsors, and university involved in the program.

1.3 Customer Needs, Projects Constraints, and Requirements

The primary need is a reliable testing system that evaluates the performance of the Buckeye AutoDrive team's autonomous driving system to ensure that it can operate smoothly across scenarios. This can be achieved through simulation testing where the group will create a variety of scenarios to test specific vehicle features. To ensure consistency, each test will be assessed using clear and measurable pass or fail criteria. The customer expects this testing system to help and support the AutoDrive team's ability to earn full points in the Testing Strategy and Scenario Variation category.

The project has several constraints with a major one being the accelerated deadline, limiting the amount of time available for scenario design and testing. The team also needs to ensure seamless cross-device compatibility. It is important that the system can run consistently on different systems used by the team and the AutoDrive group. Additionally, the team is limited by the capabilities of RoadRunner but the team must develop realistic yet challenging scenarios to represent real world variety.

As shown in Table 1, the team is expected to help the AutoDrive team score point in the AutoDrive Challenge by designing scenarios and performance metrics that effectively test the system. It is important for the team to develop a range of scenarios to test the system under a variety of conditions. Additionally, it is key that there are metric based evaluations so there can be quantifiable performance indicators to help determine pass or fail of a case. Finally, the system must visually display pass or fail outcomes for each metric. In the event of a failure, these outputs will help the AutoDrive team quickly identify the source of the issue and determine where to begin troubleshooting.

Table 1: Customer Needs and Project Engineering Requirement

Needs	Requirement	Unit	Range (Acceptable)	Ideal
Contribute Towards Testing Strategy and Scenario Variations Task MSC.1.3.2	Earn Maximum Points for Competition Section	Competition Points	15-20	20
Simulations Can Test Diverse Scenarios	Build and Autonomously Run Common Traffic Pattern Scenarios	Detailed Simulation Scenarios	10-15	15
Simulations Can Test Metrics	Detailed Metric Evaluation from Simulation Results	Individual Metrics	3-5	5

Simulation Interface	Visually Output Pass/Fail Status of Metrics	Individual Metrics	3-5	5

1.4 Use of Engineering Standards

This project does not have a physical item to make and demonstrate with, so identifying Engineering Standards does come with a challenge. However, this project is part of a competition, and with it being part of a competition, there is a rule book that was given to the team on the day the team accepted the project. Utilizing this rule book, the team was able to identify some Engineering Standards that they should use towards this project [1].

This project is currently in year 5, which is the final year of its competition. In year 4, the members of the competition were required to use the SAE J3016 standard and the ISO 21448 standard [1, p. 67]. Moving forward for year 5, the team will be continuing to use the standards SAE J3018 and SAE J3063, which including the SAE J3016 standard are the standards the members of the AutoDrive Challenge II team should follow. The specific definitions of the standards stated above are in the rule book [1, p. 88].

1.5 Social, Environmental, and/or Global Issues Impact

This project focuses on testing through simulation rather than physical testing therefore, the global and environmental impacts are low. However, the important considerations are from a social aspect of how the system developed will interact with transportation safety and autonomous vehicle development. The testing scenarios the team creates will directly affect the AutoDrive team because it will evaluate and improve their autonomous driving code. This means that the team's work will indirectly support safer autonomous vehicle behavior, which then will help protect pedestrians and other drivers. The testing done will stay within the AutoDrive team therefore, there are no privacy concerns.

In terms of legal action, since the system being created is on licensed software, such as MATLAB and RoadRunner. The team must ensure compliance with all terms of use. Even though the team is creating original work for testing scenarios, there is a small risk of infringement because there could be overlap with already patented autonomous system testing. However, because the project is for educational use and not commercial use, the likelihood of legal action is extremely low.

Since this project exists entirely online, there is a very minimal environmental impact. The system's primary impact will be the power needed to compute and run, so there is a small energy consumption impact, but it is minute compared to large companies running a supercomputer to run their simulations. The system does have the potential to have a long-range because the system could be reused and modified in the future. Since there is no physical product when the customer is done using the system, they can delete the files. There is no waste.

1.6 Research and Market Study

Research for this project was primarily conducted to select the optimal simulation environment and define the necessary analytical methods for evaluating autonomous vehicle performance. After consulting with the current AutoDrive Challenge Controls Software team and reviewing the competition rules [1], the team considered three candidate software packages: RoadRunner, CARLA, and Simulink. RoadRunner, a MathWorks product, offers realistic 3D road scenes and realistic traffic signage to create driving simulations. However, it lacks built-in, measurable feedback, and it can be time intensive to build the simulation scenes [2]. CARLA, an open-source option, allows for simulated weather conditions, but it is not user-friendly and requires powerful processing capabilities [3]. Lastly, Simulink, a MATLAB-based software, is designed for controls system modeling and hardware-in-the-loop testing. Despite these testing abilities, it provides no visual feedback and has complex testing solutions [4].

Given these considerations, the team chose to utilize RoadRunner to manage the high-fidelity simulation scene design. To compensate for Simulink's lack of data analysis, the team expanded the existing Simulink model with custom testing blocks. Further research into performance metrics for autonomous vehicles guided the selection of three core analytical blocks for the Simulink model: route tracking, lane keeping, and velocity tracking. The team determined that no existing scenes fully met the project's specific needs, leading to the development of three initial design concepts for simulated intersections: Four-Way Stop Sign Intersection, focusing on stop-line detection and right-of-way; and Four-Way Traffic Light Intersection, challenging dynamic signal detection. To provide analytic data on these simulated scenes, the team settled on route tracking, lane keep, and velocity trackers.

To objectively select the best initial scene design, the team employed a weighted decision matrix using criteria such as Ease of Implementation (30%), Relevance to Competition (40%), and Measurable Feedback Potential (30%). As seen in Table 2, The Four-Way Stop Sign Intersection achieved the highest weighted score (1.6), proving to be the most practical and relevant starting point, offering the best balance between implementation simplicity and testing fundamentals, and was thus selected as the initial focus. All information and claims regarding the software capabilities are based on research from MathWorks and the CARLA project.

Table 2: Primary Scene Decision Matrix

Concept	Implementation (Score x 0.30)	Relevance (Score x 0.40)	Feedback (Score x 0.30)	Score
Four-Way Stop	2	1	2	1.6
Four-Way Traffic Light	1	1	1	1.0

1.7 Preliminary Design

The preliminary design establishes an Automated Simulation Testing Interface (ASTI) aimed at validating vehicle control software efficiently through virtual environments. The process begins with the user interacting with a Graphical User Interface (GUI) to select and configure the desired simulation scenarios. Upon running the test via the GUI, the system initiates the Simulation & Data Acquisition Module, which automatically connects the existing Simulink car model with the RoadRunner software. RoadRunner executes the chosen simulation, recording the car model's response behavior throughout the test. This recorded data is then passed to the Data Processing & Visualization Module within Simulink, where it is analyzed and compared against predefined performance criteria to determine a definitive Pass or Fail status. Finally, these processed results, including the clear Pass/Fail notification and the option to expand and view detailed graphical representations of the data figures, are presented back to the user on the GUI. Figure 1 shows this process flow.

Figure 1: High Level Block Diagram of Solution

1.7.1 Test Scenario Construction and Performance Metric Analysis

The majority of the effort in this solution is concentrated within the Simulation and Data Processing phase. To ensure comprehensive testing, the team first constructed a robust base scene featuring standard four-way intersections. Building upon this foundational environment, the system incorporates a variety of intricate routing scenarios designed to rigorously challenge the AutoDrive car's control logic, including straight routing, left turns, and right turns. Furthermore, complexity was significantly increased by introducing dynamic obstacles for the car to evaluate, such as oncoming traffic and pedestrians. A clear example of these detailed test cases, which includes a four-way traffic light intersection and an oncoming car, can be seen in Figure 2.

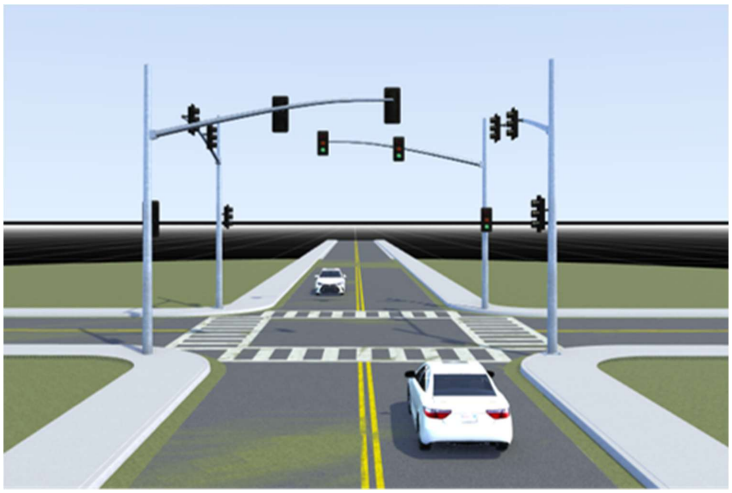

Figure 2: Four-Way Traffic Light with Oncoming Car

In addressing the complexities of the data processing phase, the team's primary focus centered on the evaluation of three critical dynamic metrics: route tracking, lane keeping consistency, and speed regulation. Meeting this challenge requires a robust method to define, capture, and compare behavior.

First, expected behavior must be precisely known for every scenario and time step. This involves setting strict performance boundaries, for instance, defining speed and trajectories for the vehicle. Second, the data needs to be translated from raw simulation output into comparable metrics. The team had to develop algorithms to accurately process the raw telemetry data output by RoadRunner. This raw data, which includes continuous streams of vehicle coordinated compared to time, must be filtered and synchronized to calculate the definitive metrics (e.g., calculating instantaneous lateral error for lane keeping). Finally, the processed data is subjected to validation criteria. If the vehicle's calculated performance metrics remain within the predefined tolerance windows for route tracking, lane keeping, and speed throughout the entire simulation run, the test registers a 'Pass' for each of these criteria. Any sustained excursion outside these bounds results in a 'Fail', ensuring that the resulting Pass/Fail notification is based on quantifiable, evidence-backed criteria.

1.7.2 Project Schedule

The project is currently active across several key work breakdown structure areas. This includes five major areas: Class Milestones, Initial Scene Building, AutoDrive Software Integration, GUI, and Data Analysis. Class Milestones included key check points throughout the project. These were the initial meetings, Product Design Review (PDR) and Conceptual Design Review (CDR). These spanned throughout much of the first semester starting September first for initial kickoff, October 20th until the 26th for PDR, and December 2nd through 8th for CDR.

The next key early milestone is the Initial Scene Building. This included building the basic Four-Way Stop Sign and Traffic Light Scenes with all varying direction scenarios. This began September 25th and finished on October 23rd. The next category was the lengthiest with any difficulties. This was the AutoDrive Software Integration which included loading the car software into the simulations from October 30th to November 12th. After completing this, the parameters identified as being used by the software needed to be adjusted in each scenario. This took place from November 11th to the 20th. The last step was to successfully run and trouble shoot the software working within the simulations. This successfully took place between November 13th to December 2nd.

The next milestone is the GUI's development. This started with the initial build design from October 25th to the 29th. From here the Start Up script to integrate with the simulation and software was created from October 31st to November 13th. Currently being wrapped up is the ability to run all selected simulations from the GUI. This was started on the 14th of November and is projected to be completed by December the 10th. The final step with the GUI will be the create an output from the simulations that provides a metric report. This was started on December 1st and will be completed by the 20th.

Before the GUI output can be completed, Data Analysis will need to be finalized. As new parameters get measured, the GUI will be updated to output this data. First Initial Research was done from October 25th to November 7th to find available simulation statistics and processing tools. From this research, the team focused on Velocity and Speed analysis from November 8th to the 21st and Comfort Metrics from November 15th to the 28th. The team is currently working on Lane Position Analysis that was started on November 20th and is projected to be completed by the 9th of December.

These dates and completion rate percentages are summarized in Table 3.

Table 3: Project Schedule

Milestone	Task	End Date	Completion Date	Completion
Class Milestones	Kick Off	9/17/25	9/17/25	100 %
	PDR	10/26/25	10/26/25	100 %
	CDR	12/08/25	12/08/25	100 %
Initial Scene Building	Create Basic Traffic Light Scene	10/04/25	10/04/25	100 %
	Create Four Way Stop Scene	10/04/25	10/04/25	100 %
	Create Left Scenarios	10/20/25	10/20/25	100 %
	Create Straight Scenarios	10/23/25	10/23/25	100 %

	Create Right Scenarios	10/30/25	10/30/25	100 %
AutoDrive Software Integration	Load Software into Simulations	11/12/25	11/12/25	100 %
	Edit Scenarios to Accommodate Software	11/20/25	11/20/25	100 %
	Run Simulations with Software	12/02/25	12/02/25	100 %
GUI	Initial Build	10/29/25	10/29/25	100 %
	Start Up Script	11/13/25	11/13/25	100 %
	Run Simulations	11/27/25	1/14/26	100 %
	Output Report	12/20/25	-	0 %
Data Analysis	Initial Research	11/07/25	11/07/25	100 %
	Velocity and Speed Analysis	11/21/25	-	75 %
	Comfort Metrics	11/28/25	11/28/25	100 %
	Lane Position Metrics	12/09/25	-	75 %

References

[1] AutoDrive Challenge II – Year 5 Ruleset, Society of Automotive Engineers, Warrendale, PA, Aug. 2024 [accessed Sept. 23, 2025].

[2] MathWorks, "RoadRunner." [Online]. Available: www.mathworks.com/products/RoadRunner.html. [Accessed: Nov. 24, 2025]

[3] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An Open Urban Driving Simulator," in Proc. 1st Conf. Robot Learning, PMLR, 2017, doi: 10.48550/ARXIV.1711.03938.

[4] MathWorks, "Simulink." [Online]. Available: https://www.mathworks.com/products/Simulink.html. [Accessed: Nov. 24, 2025]

Chapter 2: Detail Design

2.1 Scene and Scenario Creation

Using RoadRunner2024b, two primary scenes were developed: the Four-Way Traffic Light Intersection and the Four-Way Stop Sign Intersection. A comprehensive set of scenarios was designed for each scene to rigorously test the AutoDrive car against a range of real-world possibilities. Scenario variations cover critical elements like pedestrian interference, interactions with other vehicles, and standard runs without external interference. The scenarios are also differentiated by the AutoDrive car's trajectory through the intersection, encompassing tests where the vehicle drives straight, turns left, or turns right. Furthermore, specifically in the traffic light scene, scenarios were created to test reactions to different light signals: red, yellow with a stop, yellow with time to go, and green.

2.2 Graphic User Interface

The Graphical User Interface (GUI) functions as the control panel for the RoadRunner simulation environment, built using MATLAB App Designer. Its initial function is to streamline the setup process by collecting and verifying the necessary RoadRunner installation and project paths. These verified user selections are then saved in a .mat file, which establishes the critical connection between the GUI and the RoadRunner simulations. Once configured, the GUI allows the user to select specific obstacles and behavior types for the scenarios they wish to test, as illustrated in Figure 3. In addition, the GUI supports feature-based grouping, when selecting a feature it automatically selects the corresponding obstacles and behaviors, as shown in Figure 3 when selecting “Sign Detection”. After the user clicks "Run Simulation," the corresponding scenarios are executed automatically in the background.

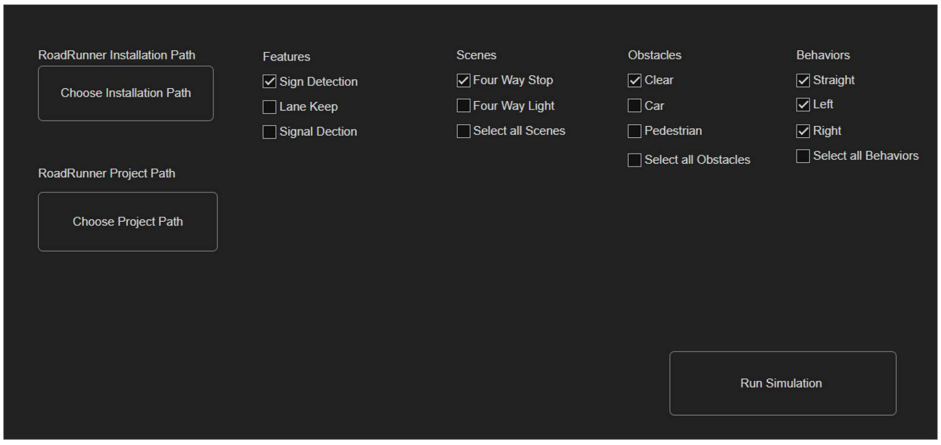

Figure 3: Graphic User Interface

The final key function of the GUI is the presentation of results: upon completion of all simulations, the collected performance metric data is processed and displayed directly on the

interface. This provides the user with immediate, clear feedback on the autonomous vehicle's performance in the simulations, as conceptually shown in Figure 4. Each scenario is first marked as pass or fail criteria. The GUI also includes graphs to let the user visually review the performance results.

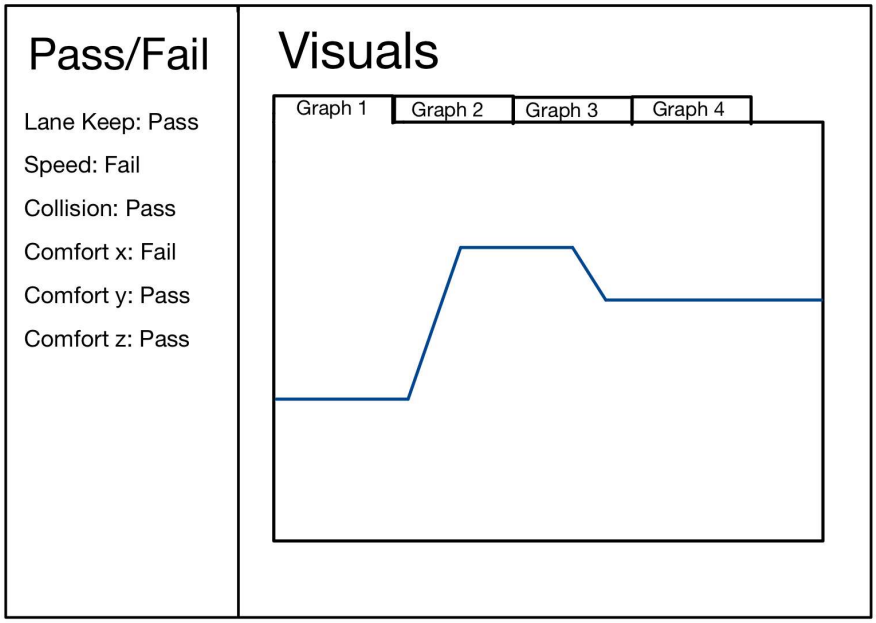

Figure 4: Output Metric Data View

2.3 Performance Metrics Framework

The performance metric framework is designed to quantitatively assess the effectiveness of the autonomous vehicle's behavior during simulation. From the created scenarios, data from the simulation is to be logged and provide objective data to evaluate how well the autonomous vehicle performs in terms of safety, comfort, and operational efficiency. The collection and analysis of data is from the use of co-simulating RoadRunner and MATLAB/Simulink. The metrics are derived from vehicle position, acceleration, velocity, and heading. Each metric has a clear threshold criterion to allow for a clear and repeatable evaluation of the autonomous vehicle data. Future iterations of this framework can incorporate additional metrics as scenario complexity increases.

2.3.1 Actor Detection and Data Extraction

To make the testing system efficient, it was important to ensure the system could test any RoadRunner scenario without prior configuration. Therefore, an actor detection and logging

system were implemented in MATLAB. Rather than the user having to hard code in how many actors there are, this system uses the SimulationLog to collect all existing actor IDs in a scenario. From there, the system will then collect data for each actor, such as their time, speed, and their x-y coordinates. All the extracted data is put into structured arrays regardless of the scenario complexity. This process enables scalability as it will work ranging from one actor to dozens depending on how complex the user wants their scene to be. This tool also eliminates the need for preprocessing, human error, and allows the user to test many scenarios quickly.

2.3.2 Visualizing the Data

After data extraction, the system will automatically generate several diagnostic visualizations that summarize the behavior of every actor in the scenario. Figure 5 shows the velocity vs time graph for the actors, where the actor's speed is computed using it's logged time and position. Plotting all actor velocities on a shared plot enables rapid inspection of behaviors such as acceleration ramps, adaptive cruise control, sudden braking, or complete stops. These metrics are crucial for validating performance to ensure compliance with speed limits or safe acceleration speed. The plot is made from the actor's data that has been stored in the arrays from before, thus will scale easily if more actors are added.

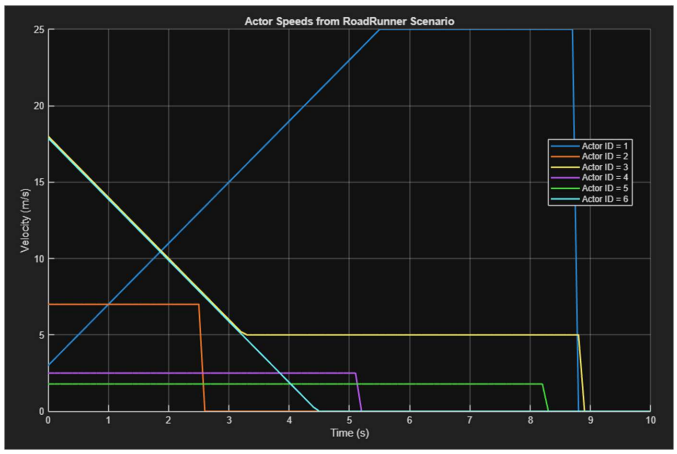

Figure 5: Velocity vs Time Graph.

In addition to the velocity vs time graph, the system also generates a 2-D map that shows each actor's path throughout the scenario. Figure 6 shows the plot that overlays all actors' positions. This visualization highlights lane keeping accuracy, intersection behavior, and potential conflicts between the AutoDrive car actor with other actors. Looking at Figure 6, it can be easy to determine if the AutoDrive code maintains proper boundaries and executes maneuvers safely.

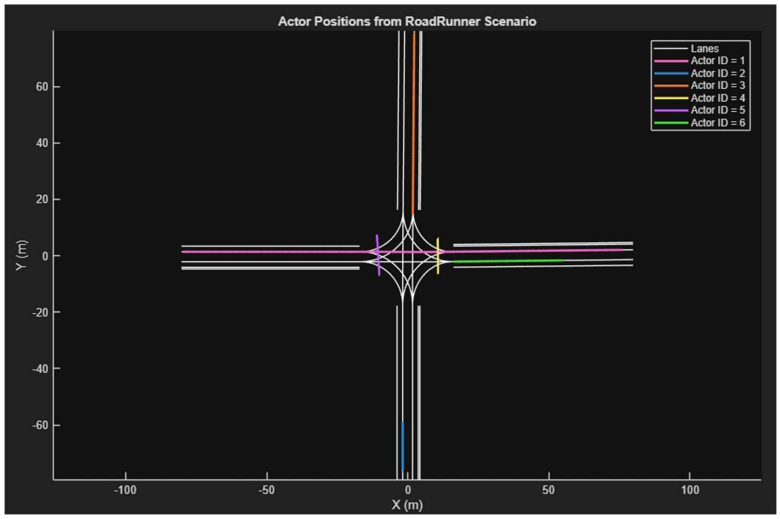

Figure 6: Birds Eye View of Scenario Plot

2.3.3 Trajectory Smoothness

Trajectory Smoothness is a critical performance metric to access the comfort and control of the autonomous vehicle system. This metric quantifies how smoothly the vehicle follows its predetermined trajectory, which directly applies to the vehicle's safety and stability. This is assessed by rate of acceleration, jerk, steering, and curvature to indicate trajectory smoothness for the autonomous vehicle's behavior. To calculate trajectory smoothness from the logged simulation data, it will be computed from longitudinal and lateral acceleration, change of acceleration, rate of change of heading angle, and time derivative of curvature. Also, the use of threshold criteria is to determine if the autonomous vehicle's behavior is smooth or unstable and this will be measured with a pass/fail criteria.

Figure 7 shows a per-actor trajectory smoothness diagnostics. It plots speed, acceleration, jerk, curvature, and curvature rate against the threshold limits to determine if the actor (autonomous vehicle) passes or fails according to the threshold criteria. Exceeding the threshold will count the test as a failure and be considered 'uncomfortable'; otherwise, it is marked as a pass.

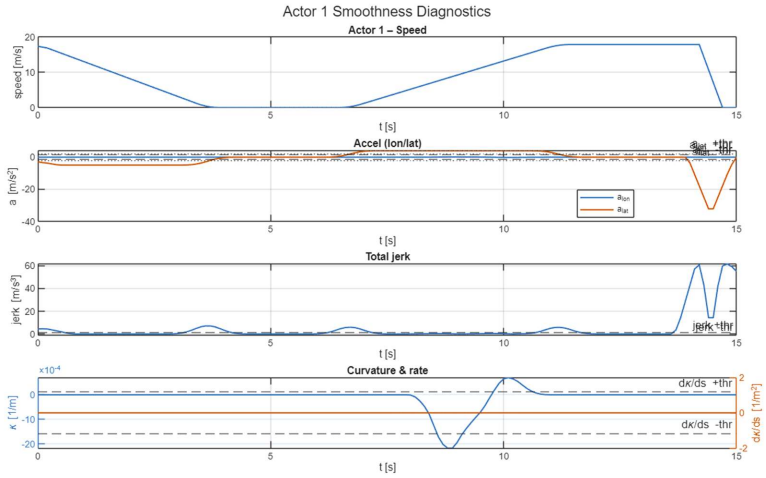

Figure 7: Trajectory Smoothness Diagnostics

Chapter 3: Prototype and Testing

3.1 Prototype

3.1.1 Start Graphical User Interface (GUI)

When users first launch the product, they are greeted by the Start GUI, seen in Figure 8, where they configure the parameters for their simulation. The process begins with the user inputting their RoadRunner installation and project paths. Next, they select their testing criteria by either choosing a predefined software feature scenario group or manually selecting individual simulations. Finally, users define the execution logic by specifying whether the tests should be repeated and if obstacle locations should be randomly generated.

Figure 8: Final Start Graphical User Interface

3.1.2 Running Simulations

Once the simulation parameters are established, the testing process executes automatically in the background. A primary MATLAB script processes the initial inputs, prepares the designated scenarios, and initializes the communication handshake with RoadRunner. To manage the execution, the script compiles all selected scenarios into a queue and processes them sequentially. For each entry, the script determines if the scenario requires randomization or can be run with standard parameters.

If the scenario does not require randomization, the script opens it directly in RoadRunner and defines the simulation run times. RoadRunner then automatically connects to Simulink to integrate the Ego AutoDrive vehicle and compile the necessary software. Once compilation is complete, the MATLAB script triggers the start of the simulation.

Conversely, if a scenario requires randomized obstacle locations, the MATLAB script opens the scenario containing only the obstacle and exports it as source code. A secondary MATLAB script then intervenes to configure the obstacle's initial position and trajectory based on randomized variables. This modified configuration is injected back into the primary Ego vehicle simulation, at which point the original MATLAB script resumes control to execute the run.

3.1.3 End Graphical User Interface (GUI)

Following the completion of the RoadRunner simulation, raw data is exported into the MATLAB workspace for post-processing. A suite of data analysis scripts dissects this information into various performance metrics, which are then converted into graphical visualizations. During this phase, the system evaluates the data against predefined safety thresholds to determine if the simulation met the required standards.

Once the analysis is complete, an End GUI launches, seen in

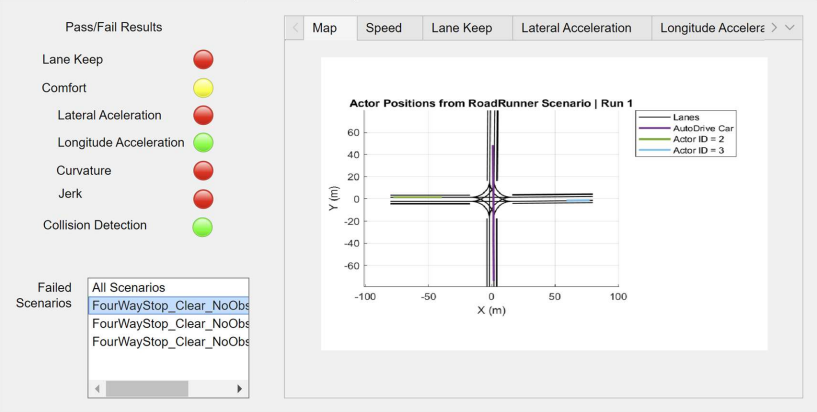

Figure 9, to provide an interactive interface for data review. This interface allows users to first evaluate performance through a system of colored status lamps: green signifies a pass for a specific metric; red indicates a failure; and denotes a partial failure within an aggregate metric evaluation. Users can then filter and view all scenarios that trigger failures or specific metrics. Finally, by selecting a scenario, users can access a tabulated window to view deeper data graphs. These plots display raw data over time alongside the relevant safety thresholds, allowing for a granular comparison of the simulation results.

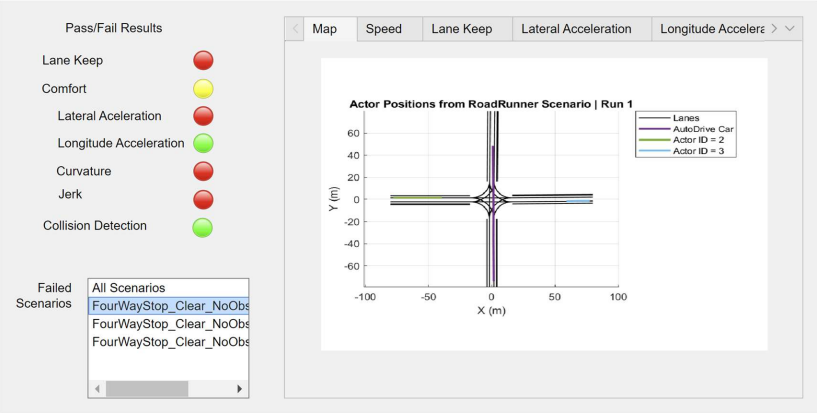

Figure 9: Final End Graphical User Interface

3.2 Testing

3.2.1 Software Test Plan

Purpose

The purpose of this test plan is to outline the process used to verify and validate the functionality of the end-to-end user framework that uses the AutoDrive's Control team software to ensure that its features and performance meet the specified requirements. These series of tests can be broken down into four defined categories: diverse scenario simulation, data metrics, input GUI, and output GUI which are used to evaluate their functionality and performance. This plan ensures that the software is tested in a controlled environment so that errors can be identified, and the overall framework can be assessed.

Scope

The scope of this test plan includes the end-to-end user framework within the four defined categories. Testing in each category is intended to be as comprehensive as possible for the current prototype. However, the tests are limited by time and the current capabilities of the RoadRunner and MATLAB/Simulink software. Factors considered during planning the series of tests included scenario availability, simulation repeatability, GUI behavior, and the reliability of generated and displayed results.

Table 4: Requirement/Verification Cross-Reference Matrix

Classes of Verification: I- First Article, II-Environmental, III-Acceptance Test, IV- None

Methods of Verification: A- Analysis, T-Test, D-Demonstration, I- Inspection

Project Requirements	Test Names	Verification Class				Test Procedures Location
		I	II	III	IV	
Diverse Scenario Simulation	1.1 Scenario Simulation	D/A		D/A		Appendix B.1.1
Data Metrics	2.1 Output Data Results	I/A		I/A		Appendix B.2.1
	2.2 Storing Data Results	I		I		Appendix B.2.2
Input GUI	3.1 Selected Scenario Simulation	D		D		Appendix B.3.1
	3.2 Repeated Scenario Simulation	D		D		Appendix B.3.2
	3.3 Set up Path Files	I/D		I/D		Appendix B.3.3
	3.4 Random Actor Generation	D		D		Appendix B.3.4
Output GUI	4.1 Output GUI Data Metric Plots	I		I		Appendix B.4.1
	4.2 Output GUI Pass/Fail Indicator	I/A		I/A		Appendix B.4.2

3.2.2 Test Explanation

1. Diverse Scenario Simulation

This test evaluates the various built scenario configurations within the RoadRunner and MATLAB/Simulink environment with an intended scripted behavior. This test is to verify whether the software runs the selected scenario behavior. For Scenario Simulation test, the purpose of this test is to verify if AutoDrive car (actor 1) in all possible RoadRunner scenario configurations created follows the intended scripted behavior during the simulation. This is a pass, if the simulated behavior of actor 1 matches the intended scripted behavior of actor 1. This is a fail if actor 1 displays a different simulated behavior than the intended scripted behavior.

2. Data Metrics

These series of tests evaluate the software's ability to generate and store the correct performance metric results with each simulation run. These tests are to verify whether the software can generate the expected metric outputs, record them correctly, and properly organize the output results for review. For output data results test, the purpose of this test is to verify that the test

framework generates the required output metric plot for each data metric and that each plot accurately represents the recorded metric data. This is a pass if all data metric plots generate and displayed with a corresponding pass/fail indicator. This is a fail if any of the data metric plots do not generate. For storing data results test, the purpose of this test is to verify the data metric plots, follow the correct naming convention, are stored in the correct file location, and are labeled with the correct pass/fail status for each run. This is a pass if all runs are stored in the runs variable with their run data correctly stored within the fails matrix. This is a fail if any of the data that should be stored inside runs or fails did not save.

3. Input UI

This next set of tests evaluates the ability of the input GUI to correctly apply user-defined simulation configurations to the software. This test is to verify whether the user interface properly configures scenario selections, run conditions, and required set up path settings, and if those selections are accurately displayed in the simulation. For selected scenario simulation test, the purpose of this test is to verify the input GUI correctly applies to the user's selected scenario options. Also, loads and runs the intended scenario configuration. This is a pass if the intended scenario loaded into the simulation and verified with the output GUI. This is a fail if the intended scenario did not load into the simulation. For repeated scenario simulation test, the purpose of this test is to verify the input GUI applies the user-selected number of runs for the chosen scenario and accurately executes the scenario the specified number of times. This is a pass if the user defined amount of runs matches the simulated amount of runs. This is a failure, if the simulation runs the incorrect amount of runs from the intended amount of runs. For set up path files test, the purpose of this test is to verify the input GUI correctly applies the selected RoadRunner installation path and project path. This is a pass if the selected installation and project paths are saved into the corresponding .txt files. This is a fail if the selected installation and project paths are not correctly saved to the corresponding .txt files. For random actor generation test, the purpose of this test is to verify the input GUI applies the random actor generation by randomly placing actor 2 in the selected scenario configuration. This is a pass when it is selected in the input GUI and actor 2 is randomly placed in the simulated scenario. This is a fail if actor 2 is not randomly placed within the simulated scenario.

4. Output UI

The final set of tests evaluate the ability of the output GUI to correctly display the output data of the simulation including metric plots and pass/fail indicators. These tests ensure that the outputted data from the output GUI is accurate with the selected run configurations. For output GUI data metric plots tests, the purpose of this test is to verify that the output UI correctly displays the generated data metric plots for the selected scenario. This is a pass if each of the data metric plots are placed in the correct metric tab on the output GUI. This is a fail if any of the data metric plots are missing or do not match the corresponding tab within the output GUI. For output GUI pass/fail indicator test, the purpose of this test is to verify that output UI correctly displays the run outcome as pass or fail for the selected scenario. This is a pass if each of the displayed pass/fail indicators correctly match the corresponding plotted data metric plot relative

to the threshold. This is a fail if the pass/fail indicator does not correctly match the data metric plot relative to the threshold.

3.2.3 Testing Results

1.1 Scenario Simulation

This test verified for all possible scenario configurations including: four-way stop and four-way light, with all obstacles: clear, car, and pedestrian, and with all behaviors: left, right, and straight, that the intended behavior of actor 1 matched the simulated behavior of actor 1. Therefore, this test passed.

2.1 Output Data Results

This test verified that in a four-way stop, no obstacle, and straight behavior scenario that all data metric plots, including map, lane keep, lateral acceleration, longitude acceleration, curvature, curvature rate, jerk, and collision detection were generated and has a pass/fail indicator associated with it. Therefore, this test passed.

2.2 Storing Data Results

This test verified that in a four-way stop, no obstacle, and straight behavior scenario that all data from the ran scenario correctly saved in runs matrix and fails matrix. Therefore, this test passed.

3.1 Selected Scenario Simulation

This test verified that in a four-way stop, no obstacle, and straight behavior scenario that the correct scenario loaded and ran as it can be seen in the failed scenario box in the output GUI. Therefore, this test passed.

3.2 Repeated Scenario Simulation

This test verified that in a four-way stop, no obstacle, and straight behavior scenario with run count of 3, the simulated scenario was repeated 3 times, resulting in 4 scenarios total in the failed scenario box. Therefore, this test passed.

3.3 Set up Path Files

This test verified that both RoadRunner installation path and project path were successfully applied and registered into the MATLAB workspace. Therefore, this test passed.

3.4 Random Actor Generation

This test verified that in a four-way stop, no obstacle, and straight behavior scenario with randomizing actor 2 in the simulation, resulted in actor 2 being randomly placed in the simulation. This required a second run to show actor 2 randomly change position. Therefore, this test passed.

4.1 Output UI Data Metric Plots

This test verified in a four-way stop, no obstacle, and straight behavior scenario that the code correctly places the generated data metric plot under the corresponding tab in the output GUI. Therefore, the test passed.

4.2 Output UI Pass/Fail Indictor

This test verified in a four-way stop, no obstacle, and straight behavior scenario that the pass/fail indicators in the output GUI correctly matched the threshold results shown in the corresponding metric plots. Therefore, this test passed.

Chapter 4: Final Design

4.1 Software Feature Scenario Grouping

Throughout the design process, the team focused on maximizing the available features to accommodate increasing testing complexity. This led to the development of software feature groupings, which enable targeted testing of specific design elements. By categorizing scenarios in this manner, the team ensures that the appropriate testing environments are pre-identified and easily accessible through the Start GUI. This implementation required extensive research into both current and upcoming software projects from the design team. Consequently, these efforts resulted in a robust scenario library capable of providing comprehensive validation for each software feature.

4.2 Automatic Simulation Execution

A primary focus of this project's development was user simplicity and autonomous capability. To meet these requirements, the team engineered a simulation execution process that functions entirely in the background, minimizing manual oversight. The automation begins by cleaning the MATLAB environment and loading the necessary system parameters and installation paths. The script dynamically adds these paths to the MATLAB environment, ensuring that the software can seamlessly access RoadRunner's application files and the specific project directory. By automating the directory management and data loading phases, the script establishes a consistent foundation for high-volume testing without requiring the user to manually configure file dependencies.

Once the environment is initialized, the script constructs a queue of testing scenarios. It iterates through combinations of scene types, obstacle behaviors, and environmental conditions to build precise file paths for each scenario file. The script opens the relevant RoadRunner project, establishes a connection to the Simulink model for the Ego AutoDrive vehicle, and configures simulation run-time variables such as step size and steering ratios. If a user has opted for randomization, the script triggers a specialized routine to inject randomized positions and trajectories into the scenario before execution.

Once finished, the script automatically triggers a series of post-processing routines, including data logging, plot generation, and safety metric evaluations. After each run, the script closes the current RoadRunner instance to clear system memory before proceeding to the next scenario in the list. This cycle repeats autonomously until all designated tests are complete, at which point the End GUI is launched to present the combined results to the user.

4.3 Data Analysis Metrics

To evaluate the performance of the autonomous driving system, the team created a structure set of data analysis metrics to quantify the vehicle behavior across all scenarios. These metrics were selected to assess safety and comfort of the vehicle while maintaining the automated simulation pipeline. For the implementation of these metrics the simulation data was collected from RoadRunner and Simulink, including vehicle position and velocity related to the simulation time.

From this data, the team created performance indicators that determined if a scenario meets predefined thresholds.

To ensure scalability and ease of analysis, the team created a structure data array that will store all computed metrics after each scenario. This data array will store scenario name, simulation data, metrics, run identification, and pass/fail results. This organization enables efficient visualization and integration with the End GUI where the results are presented to the user.

4.3.1 Trajectory Map

The trajectory map was implemented as a primary visualization tool to represent all vehicles' motion throughout each simulation. The map is created by using the position data from the simulation log. The system creates a two-dimensional plot of the vehicle's path overlaid on the roadway of the scene. This feature allows users to quickly recognize if the vehicle is following the intended path and remaining within the lane.

4.3.2 Speed Plot

The team created the speed plot generation for the user to be able to quickly visualize the vehicle's speed compared to the other actors in the scenario. The plot is created by using the time series data collected for all actors. This allows for observation of acceleration, deceleration, and steady state motion. The user can check for smooth acceleration, vehicle following speed limits, and the car avoiding unrealistic velocity changes. In addition to the visualization, the speed data will be used for other metrics to provide a more complete assessment of the autonomous driving system.

4.3.3 Lane Keep

Lane keeping was developed to verify the vehicle's ability to maintain proper alignment within its designated lane. This metric was calculated using lateral deviation meaning the vehicle's distance from the center of the lane. A predefined threshold value was determined for acceptable behavior and if the value exceeds the threshold, then a system flags the run as a failure. This metric is a critical indicator for safe driving behavior.

4.3.4 Lateral Acceleration

Lateral acceleration measures the forces the vehicle experiences during turns, therefore it is important when evaluating if the car is turning at a comfort rate for the passengers. Low lateral acceleration on curves means the autonomous driving system is turning with stability and keeping the passengers comfortable. If the value exceeds the predefined limit, then that means the vehicle performed sharp or unstable turning and the system will flag the metric as a failure for that simulation run. This metric is useful when analyzing scenes that involve tight curves or rapid direction changes.

4.3.5 Longitudinal Acceleration

Longitudinal acceleration was derived from the rate of change of the vehicle's forward velocity to calculate the acceleration and braking behavior of the vehicle. This information tells the user how the vehicle responds to changes in speed. The purpose of this metric is to ensure the safety

and comfort of the passenger. Excessive acceleration values could mean aggressive or unrealistic driving behavior and the metric would be flagged as a failure for that simulation run.

4.3.6 Curvature

To evaluate how the vehicle responds to turns and curves the team created the curvature metric. This metric represents the rate at which the vehicle's trajectory changes, providing a measurement at how sharply the vehicle turns at any given moment. This provides insight into the smoothness and controlled steering behavior of the vehicle. If the value is above the threshold value, then it indicates the vehicle is overly aggressive when turning and could suggest understeering or a delayed response.

4.3.7 Curvature Rate

From the curvature metric curvature rate can be derived which is calculated by the rate of change of curvature. This is important because this shows how quickly the vehicle transitions between different turning states. High curvature rate could mean instability or abrupt steering from the autonomous driving system thus showing the importance of this metric. If the value is above the threshold, then the metric will be flagged as a failure for the simulation run.

4.3.8 Jerk

Jerk is an additional metric that evaluates the smoothness of the vehicle's motion through the rate of change of acceleration over time. This metric provides data showing the vehicle's transitions between different states of motion. Even if the acceleration metrics fall within acceptable limits, high jerk values can be found indicating poor ride quality. Jerk allows the user to evaluate the smoothness of driving.

4.3.9 Collision Detection

Collision detection was implemented as a critical metric for safety. Not only does this metric determine if the vehicle crashes into another actor but determines how close they encounter an actor during a simulation. This metric is derived by using the actors' position and determining the distance between them. If the vehicle gets within 3 feet of another actor, then the system marks the simulation as a failure. Therefore, one of the most important metrics when determining safety of the passenger.

4.3.10 Comfort

The comfort metric was developed as an overall evaluation of the autonomous driving system for each simulation run. Unlike the other metrics that focus on one area this metric looks at the comfort metrics that include longitudinal and lateral acceleration, jerk, curvature, and curvature rate. If all the metrics fail then the comfort is marked as red. If one or more metrics fail then comfort is marked as yellow and if they all pass then comfort is marked as green. Comfort takes multiple dynamic behaviors and combined them into an easy result to interpret. This allows users to quickly identify the overall driving quality without having to look through all the metrics.

4.4 Random Obstacle Generator

To better the simulations the team decided that random obstacle generator would improve testing as it would increase variability. This allows for the autonomous driving system to be tested with more conditions without the need for more scenario files to be created. The random obstacle generator operates as a routine before a simulation begins. On the Input GUI the user is given the option to check a box for randomize obstacles. If this box is checked then in the StartUp file after the scenario name has been set it will be changed to a file specially created for random actor generation. This RoadRunner file ends in “RAG” (Random Actor Generator) and has actor’s two and three in it and will be opened. Then the RandomActorGeneration Matlab file is called. This script will export the data from the RAG file. From there the actor two’s position can be accessed and will be randomly selected to a value between predefined bounds. Then the RAG RoadRunner file will be close and a new RoadRunner file will open. The new RoadRunner file is called the scenario name with “Blank” at the end of it. This file only contains actor one with the autonomous driving system’s behavior connected to it. Next, the Matlab code will import the new position of actor two and actor three into the scenario. Once everything has successfully loaded the code will return to the StartUp file. The code and the RoadRunner simulation will then run as normal. This process will occur for each repeated simulation randomly moving actor two. This changes the timing of the interactions between actor one and two and will test new behaviors.

Commented [WS1]: Dive into how the code works in detail. No need for screenshots, but a written walk through as if the reader has the code open with them

Chapter 5: User Manual

5.1 Introduction

5.1.1 What is Simulation?

Simulation is the backbone of autonomous systems development, acting as a virtual proving ground that mirrors the complexities of real-world environments. In the context of the Buckeye AutoDrive project, it is a controlled digital space where team uses tools like RoadRunner and MATLAB to model intricate driving scenes, traffic patterns, and weather conditions. By creating a seamless connection between the vehicle's control software and these virtual scenarios, simulation allows for the testing and observation of autonomous behaviors, such as perception and lane keeping, without the need for a physical vehicle.

5.1.2 Why is it Important?

Simulation is important because it enables rigorous software testing while eliminating the high costs and safety risks associated with physical trials on actual roads. For the AutoDrive team, this virtual testing is essential for validating software performance through repeatable, diverse scenarios and resulting performance metrics. Furthermore, by incorporating innovations like random obstacle generation, simulation allows developers to identify and address "edge case" scenarios and software "hiccups" in a safe space. Ultimately, this data-driven validation ensures that the autonomous system is both reliable and safe before it is eventually translated to the physical vehicle.

5.2 Product Description and Goal

5.2.1 Key Requirements

There were four main needs that we wanted to address from doing this project. These needs are located on the requirements table that is on page 8 in this document. For the first need, it establishes the goal of our project to our team, which is to do the work that is listed in the remaining three needs to do well when it is time to present our project to the competition judges.

Our project satisfies the second need by the team, creating the two driving scenes and the seventy-five total different scenarios. The two driving scenes that were created were a Four Way Stop Sign Scene and a Four Way Traffic Light Scene. There were fifteen driving scenarios for the Four Way Stop Sign Scene and sixty driving scenarios for the Four Way Traffic Light Scene.

The third need from the requirements table is satisfied by our team by establishing the certain metrics we found the most important to show in our project's results. The metrics we decided to include and show in our end graphical user interface (GUI) were lane keep, longitudinal acceleration, lateral acceleration, curvature, curvature rate, jerk, collision detection, and comfort. The results showed if the metric was successful or not and there was also a output plot for some of the metrics.

For the fourth need to be satisfied, the output data was shown in the end GUI. Either red, yellow, or green is shown, which helps the user to identify if the results of metric being shown were completely unsuccessful, partially unsuccessful, or completely successful, respectively.

5.2.2 Technologies Used

The project uses several software tools to create, run, and analyze autonomous vehicle simulations. The main development tool is MATLAB R2024b, MATLAB is used to run simulations as well as integration with other software tools. Simulink is used to model the vehicle control system and run simulations models that interact with software environments.

The project also uses RoadRunner, which is an interactive scene editor used for simulating and testing autonomous driving. RoadRunner is integrated with Simulink via co-simulation, allowing the vehicle model to interact with the simulated environment in real time.

The graphical user interface for running simulations and viewing results was built using MATLAB App Designer. It allows users to select scenarios, configure simulation options, and visualize performance metrics.

Lastly, Github was used to manage code updates and maintain project repository.

5.2.3 Repository and Folder Structure

The project repository AutoDrive-RoadRunner contains the simulation framewoerk used for the AutoDrive Challenge testing environment.

The main script, StartUp.m, sets up the environment and executes the simulations. The Misc Models folder contains simulink models, while scenarios and related files are stored across the repository. Sensor Configs and Vehicle Dynamics folder define sensor settings and vehicle models, and Helper Fns include supporting scripts. ADC_RoadRunner.slxc connects RoadRunner with Simulink.

As shown in Figure 10, the repository is organized to make the system easier to edit and manage.

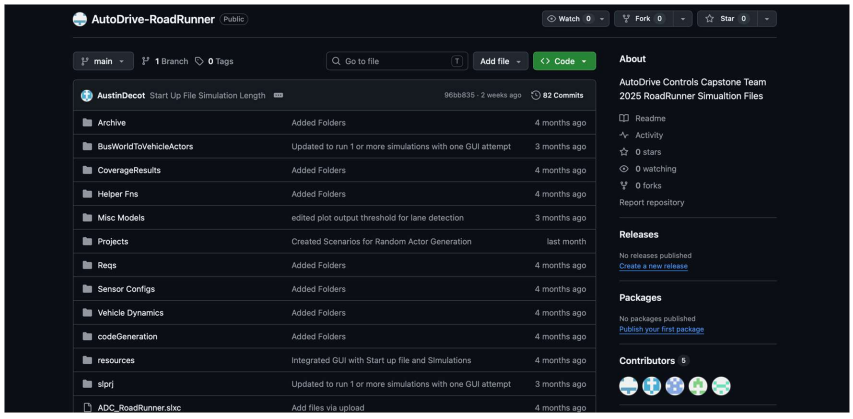

Figure 10: Part of AutoDrive-RoadRunner Github repository

5.3 How Things Work: Code

5.3.1 Graphical User Interfaces

The system includes two main GUIs developed in MATLAB App Designer, to manage simulation setup and visualize results.

The Start GUI starts with verifying the user's RoadRunner Installation and Project Paths. After that, the user selects the simulation parameters: the scenes, obstacles, and behaviors. It also supports feature-based grouping, which auto selects parameters related to the feature. Once parameters are chosen, the run button will pass the inputs to backend script to automatically run the selected simulations.

The GUI is designed so new simulation options can be added without the need to create a new interface. To add a new simulation option, you should go to Design View and add the checkboxes wanted in the Start GUI interface. The option name must match the correct name in the .rrscenario file. The corresponding scenario file must be in the RoadRunner project folder. Once added, the StartUp.m file will automatically recognize the new simulation option.

To add new feature-based grouping, add a new feature checkbox in Design View and link it to the predefined combination of scenes, obstacles, and behaviors in the Code View. When selected, the feature grouping automatically selects all associated parameters.

The End GUI displays the simulation results after execution. It evaluates performance metrices and presents them using a color-coded system: green for pass, red for fail, and yellow for mixed pass/fail for Comfort Metric, which are Jerk, Curvature, Curvature rate, Lateral Acceleration and Longitudinal Acceleration. The interface also provides plots, allowing the user to view vehicle behavior and analyze results.

Together, these GUIs form a user interaction pipeline, having a smooth transition from setting up simulation to viewing results. The interfaces reduce manual effort while improving usability and results analyzing results more efficiently.

5.3.2 Scenes and Scenarios

For each scene you want to build you first download and launch RoadRunner2024b and click on new project. You then create a name for the new project and specify the project file path to the team's entire GitHub project file. Within the GitHub file, the file path you go to is you click on the AutoDrive-RoadRunner file and then you click on the Projects file. This is where each new scene you create will be saved.

Commented [MS2]: How to build new ones and add them to the project structure

After you are done creating your new scene on the top right side of the program you switch from scene editing to scenario editing. From there you can create as many scenarios you want for the scene. The file the scenarios are all saved into is within the project file you created in the previous paragraph. Specifically, you click on the project file you created and then go into the scenarios file.

Any change you make in the scene or scenario will automatically be registered by GitHub. For every change you make you will commit the changes and then push to origin on GitHub.

5.3.3 Data Metrics and Graphs

After each simulation run the MATLAB code will pull the Simulation Log from RoadRunner. From the Simulation Log the team has created the MATLAB code to calculate each of the metrics that were mentioned in Section 4.3. This information is then stored in a structured array called “runs.” The Runs will hold the Run ID, scenario name, total actors, results, and binary fail flag. If any metric failed, then the run will be marked as a failure. All failed runs will be put into an array called “fails.” The fails array is organized by Run ID, scenario name, results, and a binary fail flag for each of the metrics to easily identify trends.

In addition to the numerical results, the MATLAB code generates a series of plots that visually represent the data. Each of the metrics have been broken down into separate MATLAB scripts. The StartUp file will call each script and in the script the metric will be calculated, stored, and plotted. If any new metrics are to be created all that is needed is a new MATLAB script that can be called from the StartUp file and called under the other metric files. The new file will need to follow the same naming conventions as the other metric files and plots will need to be stored the same way so then the End GUI can find and display the plots. For the plots to be displayed the End GUI code will have to be updated.

5.4 Trouble Shooting and Common Issues

5.4.1 Simulation Crashing

The biggest cause for a simulation to crash is that MATLAB can only connect to one RoadRunner instance at a time. If there is a RoadRunner window open when a simulation starts, then the simulation will crash and give an error. Therefore, before running any simulation ensure that all RoadRunner windows are closed to avoid this error. If you do run into this error just close all RoadRunner windows and run the simulation again.

5.4.2 Simulation Handshake Fails

The most common reason for this handshake failing other than a crash mentioned above is when more than one instance of RoadRunner is open. Before attempting to run a new simulation, the user should close all instances of RoadRunner that are open. This allows for a clear slate. To identify if this is the issue other than simulations not running, the user can go to the MATLAB Command window and see a comment regarding open RoadRunner instances.

Chapter 6: Project Management, Summary, and Conclusion

6.1 Project Management

6.1.1 Management Strategy and Tools

The management strategy for the Buckeye AutoDrive Testing Strategy and Scenario Variation project utilized lead-focus with cross-functional integration. While each team member was assigned as a subject matter expert, the workflow encouraged significant collaboration to ensure seamless system integration. Walker served as the project lead, overseeing autonomous execution and software integration. Stanley managed scene and scenario architecture, while Dylan directed the data analytics suite. Austin led advanced feature design, including random actor generation and file storage protocols, and Khalid oversaw the development of the user interfaces.

To facilitate collaboration and version control, the team employed GitHub for all software sharing and iterative development. The technical stack was primarily restricted to the MathWorks software suite, while project management and administrative tasks were centralized within Microsoft Office. We utilized Gantt charts for timeline management and maintained a centralized location for meeting notes and action items to ensure accountability.

6.1.2 Technical Limitations and Lessons Learned

Technical limitations were identified early in the development cycle, most notably regarding software version compatibility. While the 2025 MathWorks suite was initially preferred, the team discovered critical functional gaps in the "handshake" between MATLAB and RoadRunner within that version. To mitigate this risk, the team made the strategic decision to downgrade to the 2024 release. Although this slightly limited native feature availability provided a stable, proven base model. The team successfully developed custom workarounds for missing features, ensuring all project requirements were met without sacrificing stability.

Regarding management risks, the team initially encountered challenges with time management and task completion. To address this, the team implemented a more rigorous internal schedule with "buffer dates" set significantly ahead of formal deadlines. This provided a margin for troubleshooting unforeseen design flaws. Additionally, the team transitioned from large, milestone assignments to a sprint-based approach, breaking down complex tasks into smaller, manageable deliverables. This shift improved individual task completion and allowed for more frequent integration checks throughout the project lifecycle.

6.2 Summary

Throughout the project's lifecycle, several critical milestones were achieved to transform manual, in-car testing into an autonomous validation pipeline. The first iteration featured environment stabilization, successfully resolving software versioning conflicts to establish a functional handshake between MATLAB 2024 and RoadRunner. It also included initial scene and scenario development in RoadRunner. The second iteration focused on advanced feature development. This encompassed the implementation of the automated simulation execution and the random actor generation script. The last iteration featured the End GUI tool that required complex data

analysis and visualization. The final interface allowed users to interact with the simulation runs and evaluate their success.

The developed solution solves the presented problem by automating the Buckeye AutoDrive testing workflow, providing a robust library of scenario variations. By integrating metric evaluations and automated pass/fail indicators, the system has streamlined the development cycle. The final solution allows for comprehensive testing of the AutoDrive vehicle software across a diverse array of real-world simulations and edge cases.

6.3 Conclusion

Moving forward, the team recommends prioritizing the variety and depth of testability. While the current solution provides a stable and autonomous foundation for software validation, future iterations should focus on diversifying the scenario library to include more complex driving environments. This can include roundabouts, highways, and high-pedestrian zones. Further complexity could also be added by diversifying obstacles and behaviors. These developments would further test the AutoDrive vehicle's capabilities without the requirement of on-road testing.

Lastly, as MathWorks continues to develop new versions of software, it is encouraged that future teams try to integrate new features. In completing this, the value of a solid handshake needs to be stressed as the backbone of a successful system.

Dylan Vallen: During this project, I was primarily tasked with the data analysis steps. This required a lot of critical thinking and creativity to take the basic raw outputs and process it into a variety of different metrics. Much of this thinking was initially done early in the project. During this time, I learned a lot about what questions to ask and how to gain clarity when confused. I also learned the importance of making sure our sponsor/customer was on the same page as the team. This was a critical failure that the team and I worked on moving forward, and I gained a lot of experience and exposure through this.

Walker Sheidow: As the project manager and lead, I supported the coordination and communication of the team. My primary contributions were in organizing the team as well as being the primary support for MathWorks suite tools. This includes development of the early Start GUI and the final End GUI. I also co-developed Lane Keep metrics in Simulink and presented at the team competition in Detroit, MI.

With extensive previous knowledge on the topic, I supported the learning of other members whose tasks relied on MathWorks systems. I learned a lot about knowledge sharing through this process. These are complex, technical tools that can be challenging to understand initially, especially without much background knowledge. This provided difficulties in learning, and I was challenged to present information in diverse ways to support different learning styles. I increased my skills in patience and empathy through this process by seeing the world from other's views and supporting their learning.

Stanley Mattam: Throughout the capstone semesters I was focused on building and continuously improving on the two scenes and seventy-five scenarios that were built. For each idea we had for the scenes/scenarios, I made sure to apply that idea to the all the scenes/scenarios

we wanted to complete for competition. Some of these ideas included setting the speed of the pedestrian to 2 mph for all the scenarios that had a pedestrian in it, making the AutoDrive vehicle start from south of the intersection and go north to then display its intended scenario, etc.

The biggest takeaway for me from this project was the importance of calling upon others for help when you are unsure what to do. I first got tasked with exploring the idea of the Random Actor Generation. When my initial research proved that this is something that I will need additional help on to move forward, I then called Austin and Walker and filled them in on my initial research. The result of me doing this led to Austin further exploring the Random Actor Generation idea and making it come to life. Before this I would have been a lot more hesitant to ask for help but now this concept of asking for help is something I will confidently take into my future job whenever I am unsure what to do. Being a successful engineer in the field consists of continuously learning from others. I would definitely say I am no longer hesitant in making sure I ask the right questions and get the right help from the right people.

Khalid Azzabin: My main part of this project was the development of the start and end GUIs and their integration with MATLAB, Simulink, and RoadRunner. I focused on creating a workflow that allows users to easily configure simulations, run them, and visualize results. I faced technical challenges related to co-simulation and data extraction, which needed debugging and coordination with the team. Ultimately, this project improved my skills in research, debugging, and developing autonomous vehicle simulations.

Austin Decot: My main contribution to this project has been focused on developing data metrics, analysis, visualization tools, and random actor generation to help evaluate the autonomous driving system. I did this by designing a structured and organized system to store all simulation data and sorting plots into pass or fail cases. This was essential for the GUI to be able to easily access the information as well as a great tool for the user to quickly find specific information. This project allowed me to strengthen my skills in MATALB, Simulink, and system debugging, particularity in the integration of RoadRunner with the automated data processing. The biggest challenge I faced was figuring out how to implement random actor generation with RoadRunner as it required working around limitations. I addressed tis by experimenting with scenario file manipulation and refining actor placement. This process improved my critical thinking, troubleshooting complex systems, and adapting to changes within the system. This project overall made me feel more confident and effective as an engineer.

Appendix B – Testing Procedures

B.1.1 Scenario Simulation Test	
Method of Verification: Demonstration/Analysis	
Type of Test: First Article/Acceptance Test	
Date(s)/ Total Times: 3/27/2026	Location: Online
Test Equipment: Example: Computer, RoadRunner 2024b License, MATLAB 2024b License, AutoDrive-RoadRunner GitHub repository	
Test Engineer: Stanley Mattam **Witness:** Dylan Vallen	
Criteria for Success: ☒ Passes – For all possible scenarios, if the name of the behavior in the selected scenario name matches the behavior of actor 1 in the corresponding map plot ☐ Fails – If actor 1 displays a different behavior in the map plot compared to the behavior in the scenario name	
Test Procedure: 1. Access the AutoDrive-RoadRunner GitHub repository and download the files 2. Run MATLAB 2024b and open AutoDrive-RoadRunner\RR_GUI.mlapp 3. Run RR_GUI.mlapp and Click – Choose Installation Path 4. Select your RoadRunner Installation Path through your file explorer, the file should end with \RoadRunner R2024b\bin\win64 5. Click – Choose Project Path 6. Select your RoadRunner Project Path through your file explorer – the needed file is AutoDrive-RoadRunner 7. Select scenes – Four Way Stop and Four Way Light 8. Select obstacles – Clear, Car, and Pedestrian 9. Select Behaviors – Select all Behaviors 10. Click – Run Simulation 11. Once the output GUI populates, go through each scenario in the failed scenario box and compare name of scenario to corresponding map plot of actor 1	

Test Results: See Appendix B – Testing Results: 1.1. Scenario Simulation Test Validation Screenshot (1-75)	
Action Items: N/A	
Test Engineer: Stanley Mattam, 3/27/2026	Witness: Dylan Vallen, 3/27/2026

B.2.1 Output Data Results Test	
Method of Verification: Inspection/Analysis	
Type of Test: First Article/Acceptance Test	
Date(s)/ Total Times: 3/30/2026	Location: Online
Test Equipment: Example: Computer, RoadRunner 2024b License, MATLAB 2024b License, AutoDrive-RoadRunner GitHub repository	
Test Engineer: Dylan Vallen **Witness:** Austin Decot	
Criteria for Success: ☒ Passes – If all Map, Speed, Lane Keep, Lateral Acceleration, Longitude Acceleration, Curvature, Curvature Rate, Jerk, and Collision Detection plots generated. The data from the plots displays correctly to the corresponding pass/fail indicator ☐ Fails – If any of the data metric plots do not populate or the data from the plots does not display correctly with the corresponding pass/fail indicator	
Test Procedure: 1. Access the AutoDrive-RoadRunner GitHub repository and download the files 2. Run MATLAB 2024b and open AutoDrive-RoadRunner\RR_GUI.mlapp 3. Run RR_GUI.mlapp and Click – Choose Installation Path 4. Select your RoadRunner Installation Path through your file explorer, the file should end with \RoadRunner R2024b\bin\win64 5. Click – Choose Project Path 6. Select your RoadRunner Project Path through your file explorer – the needed file is AutoDrive-RoadRunner 7. Select a scene – Four Way Stop, an obstacle – Clear, and a behavior – Straight 8. Click – Run Simulation 9. After the run completes the output GUI will populate, in the failed scenarios box click – FourWayStop_Clear_NoObstacle_Striaght	

10. Analyze the data from the Lane Keep, Lateral Acceleration, Longitude Acceleration, Curvature, Curvature Rate, Jerk, and Collision Detection plots display the correct corresponding pass/fail (green/red light) indictor.	
Test Results: See Appendix B – Testing Results: 2.1. Output Data Results Test Validation Screenshot (1-9)	
Action Items: N/A	
Test Engineer: Dylan Vallen, 3/30/2026	Witness: Austin Decot, 3/30/2026

B.2.2 Storing Data Results Test	
Method of Verification: Inspection	
Type of Test: First Article/Acceptance Test	
Date(s)/ Total Times: 3/30/2026	Location: Online
Test Equipment: Example: Computer, RoadRunner 2024b License, MATLAB 2024b License, AutoDrive-RoadRunner GitHub repository	
Test Engineer: Austin Decot **Witness:** Dylan Vallen	
Criteria for Success: ☒ Passes – If all runs are stored in the runs variable with their data of RunID, scenario name, number of actors, all results data, and the fail flag stating if a run failed or not. As well as if all failed runs were properly stored inside the fails variable that stores the RunID, scenario name, number of actors, results, and marks which metrics failed in the scenario. ☐ Fails – If any of the data that should be stored inside the runs or fails variable did not save	
Test Procedure: 1. Access the AutoDrive-RoadRunner GitHub repository and download the files 2. Run MATLAB 2024b and open AutoDrive-RoadRunner\RR_GUI.mlapp 3. Run RR_GUI.mlapp and Click – Choose Installation Path 4. Select your RoadRunner Installation Path through your file explorer, the file should end with \RoadRunner R2024b\bin\win64 5. Click – Choose Project Path 6. Select your RoadRunner Project Path through your file explorer – the needed file is AutoDrive-RoadRunner 7. Select a scene – Four Way Stop, an obstacle – Clear, and a behavior – Straight - Repeat Count 2 8. Click – Run Simulation 9. After the run completes, open MATLAB workspace	

10. Open variable “runs” to see all ran scenarios 11. Open variable “fails’ to see all scenarios that failed	
Test Results: See Appendix B – Testing Results: 2.2. Storing Data Results Test Validation Screenshot (1-2)	
Action Items: N/A	
Test Engineer: Austin Decot, 3/30/2026	Witness: Dylan Vallen, 3/30/2026

B.3.1 Selected Scenario Simulation Test	
Method of Verification: Demonstration	
Type of Test: First Article/Acceptance Test	
Date(s)/ Total Times: 3/30/2026	Location: Online
Test Equipment: Example: Computer, RoadRunner 2024b License, MATLAB 2024b License, AutoDrive-RoadRunner GitHub repository	
Test Engineer: Khalid Azzabin **Witness:** Walker Sheidow	
Criteria for Success: ☒ Passes – If in the failed scenario box, FourWayStop_Clear_NoObstacle_Striaght populates ☐ Fails – If in the failed scenario box, FourWayStop_Clear_NoObstacle_Striaght does not populate	
Test Procedure: 1. Access the AutoDrive-RoadRunner GitHub repository and download the files 2. Run MATLAB 2024b and open AutoDrive-RoadRunner\RR_GUI.mlapp 3. Run RR_GUI.mlapp and Click – Choose Installation Path 4. Select your RoadRunner Installation Path through your file explorer, the file should end with \RoadRunner R2024b\bin\win64 5. Click – Choose Project Path 6. Select your RoadRunner Project Path through your file explorer – the needed file is AutoDrive-RoadRunner 7. Select a scene – Four Way Stop, an obstacle – Clear, and a behavior – Straight 8. Click – Run Simulation 9. After the run completes the output GUI will populate, confirm that FourWayStop_Clear_NoObstacle_Striaght was executed in the failed scenario box	

Test Results: See Appendix B – Testing Results: 3.1. Selected Scenario Simulation Test Validation Screenshot	
Action Items: N/A	
Test Engineer: Khalid Azzabin, 3/30/2026	Witness: Walker Sheidow, 3/30/2026

B.3.2 Repeated Scenario Simulation Test	
Method of Verification: Demonstration	
Type of Test: First Article/Acceptance Test	
Date(s)/ Total Times: 3/30/2026	Location: Online
Test Equipment: Example: Computer, RoadRunner 2024b License, MATLAB 2024b License, AutoDrive-RoadRunner GitHub repository	
Test Engineer: Stanely Mattam **Witness:** Dylan Vallen	
Criteria for Success: ☒ Passes – If in the failed scenario box there are 4 FourWayStop_Clear_NoObstacle_Striaght options to click ☐ Fails – If in the failed scenario box there are not exactly 4 FourWayStop_Clear_NoObstacle_Striaght options to click	
Test Procedure: 1. Access the AutoDrive-RoadRunner GitHub repository and download the files 2. Run MATLAB 2024b and open AutoDrive-RoadRunner\RR_GUI.mlapp 3. Run RR_GUI.mlapp and Click – Choose Installation Path 4. Select your RoadRunner Installation Path through your file explorer, the file should end with \RoadRunner R2024b\bin\win64 5. Click – Choose Project Path 6. Select your RoadRunner Project Path through your file explorer – the needed file is AutoDrive-RoadRunner 7. Select a scene – Four Way Stop, an obstacle – Clear, and a behavior – Straight 8. In the Repeat Count Box enter “3” 9. Click – Run Simulation 10. After the run completes the output GUI will populate, confirm in the failed scenario box, FourWayStop_Clear_NoObstacle_Striaght was executed 4 times	

Test Results: See Appendix B – Testing Results: 3.2. Repeated Scenario Simulation Test Validation Screenshot	
Action Items: N/A	
Test Engineer: Stanley Mattam, 3/30/2026	Witness: Dylan Vallen, 3/30/2026

B.3.3 Set up Path Files Test	
Method of Verification: Inspection/Demonstration	
Type of Test: First Article/Acceptance Test	
Date(s)/ Total Times: 3/30/2026	Location: Online
Test Equipment: Example: Computer, RoadRunner 2024b License, MATLAB 2024b License, AutoDrive-RoadRunner GitHub repository	
Test Engineer: Walker Sheidow **Witness:** Dylan Vallen	
Criteria for Success: ☒ Passes – If the selected paths are registered into the corresponding .txt files - Installation and Project path saved to: • SelectedInstallationPath.txt • SelectedProjectPath.txt ☐ Fails – If the following messages appear in the MATLAB command window: • Installation folder selection cancelled. • Project folder selection cancelled.	
Test Procedure: 1. Access the AutoDrive-RoadRunner GitHub repository and download the files 2. Run MATLAB 2024b and open AutoDrive-RoadRunner\RR_GUI.mlapp 3. Run RR_GUI.mlapp and Click – Choose Installation Path 4. Select your RoadRunner Installation Path through your file explorer, the file should end with \RoadRunner R2024b\bin\win64 5. Click – Choose Project Path 6. Select your RoadRunner Project Path through your file explorer – the needed file is AutoDrive-RoadRunner 7. In the MATLAB command window check for output message if path files were registered	

Test Results: See Appendix B – Testing Results: 3.3. Set up Path Files Test Validation Screenshot	
Action Items: N/A	
Test Engineer: Walker Sheidow, 3/30/2026	Witness: Dylan Vallen, 3/30/2026

B.3.4 Random Actor Generation Test	
Method of Verification: Demonstration	
Type of Test: First Article/Acceptance Test	
Date(s)/ Total Times: 3/30/2026	Location: Online
Test Equipment: Example: Computer, RoadRunner 2024b License, MATLAB 2024b License, AutoDrive-RoadRunner GitHub repository	
Test Engineer: Austin Decot **Witness:** Stanley Mattam	
Criteria for Success: ☒ Passes – Actor 2 is randomly placed in the user selected scenario, and the scenario can run without causing errors. ☐ Fails – Actor 2 cannot be randomly placed in the user selected scenario, and/or the scenario cannot run without causing errors.	
Test Procedure: 1. Access the AutoDrive-RoadRunner GitHub repository and download the files 2. Run MATLAB 2024b and open AutoDrive-RoadRunner\RR_GUI.mlapp 3. Run RR_GUI.mlapp and Click – Choose Installation Path 4. Select your RoadRunner Installation Path through your file explorer, the file should end with \RoadRunner R2024b\bin\win64 5. Click – Choose Project Path 6. Select your RoadRunner Project Path through your file explorer – the needed file is AutoDrive-RoadRunner 7. Select a scene – Four Way Stop, an obstacle – Car, a behavior – Left, and Repeat Count 1 8. Click the check box “Randomize Obstacles” 9. Click – Run Simulation 10. After the run completes the output GUI will populate 11. Analyze the map plot and identify the colored line associated with actor 2 in each of the plots	

12. After looking at the map plot determine if the starting position of actor 2 changes for each plot, which will indicate a successful completion of this test	
Test Results: See Appendix B – Testing Results: 3.4. Random Actor Generation Test Validation Screenshot (1)	
Action Items: N/A	
Test Engineer: Austin Decot, 3/30/2026	Witness: Stanley Mattam, 3/30/2026

B.4.1 Output GUI Data Metric Plots Test	
Method of Verification: Inspection	
Type of Test: First Article/Acceptance Test	
Date(s)/ Total Times: 3/31/2026	Location: Online
Test Equipment: Example: Computer, RoadRunner 2024b License, MATLAB 2024b License, AutoDrive-RoadRunner GitHub repository	
Test Engineer: Dylan Vallen **Witness:** Walker Sheidow	
Criteria for Success: ☒ Passes – If each of data metric plot title correctly matches the metric title on the output GUI • Metric title: Map to data metric plot: Actor Positions • Metric title: Speed to data metric plot: Actor Speeds • Metric title: Lane Keep to data metric plot: Position Error vs Time • Metric title: Lateral Acceleration to data metric plot: Lat Accel • Metric title: Longitude Acceleration to data metric plot: Long Accel • Metric title: Curvature to data metric plot: Curvature • Metric title: Curvature Rate to data metric plot: Curvature Rate • Metric title: Jerk to data metric plot: Jerk • Metric title: Collision Detection to data metric plot: Actor 1 to Actor 2 Distance ☐ Fails – If each of data metric plot title does not correctly match the metric title on the output GUI	
Test Procedure: 1. Access the AutoDrive-RoadRunner GitHub repository and download the files 2. Run MATLAB 2024b and open AutoDrive-RoadRunner\RR_GUI.mlapp 3. Run RR_GUI.mlapp and Click – Choose Installation Path 4. Select your RoadRunner Installation Path through your file explorer, the file should end with \RoadRunner R2024b\bin\win64	

5. Click – Choose Project Path 6. Select your RoadRunner Project Path through your file explorer – the needed file is AutoDrive-RoadRunner 7. Select a scene – Four Way Stop, an obstacle – Clear, and a behavior – Straight 8. Click – Run Simulation 9. After the run completes the output GUI will populate, confirm that each data output metric plot title corresponds to the correct metric title on the output GUI	
Test Results: See Appendix B – Testing Results: 4.1. Output UI Data Metric Plots Test Validation Screenshot (1-9)	
Action Items: N/A	
Test Engineer: Dylan Vallen, 3/31/2026	Witness: Walker Sheidow, 3/31/2026

B.4.2 Output GUI Pass/Fail Indicator Test	
Method of Verification: Inspection/Analysis	
Type of Test: First Article/Acceptance Test	
Date(s)/ Total Times: 3/31/2026	Location: Online
Test Equipment: Example: Computer, RoadRunner 2024b License, MATLAB 2024b License, AutoDrive-RoadRunner GitHub repository	
Test Engineer: Walker Sheidow **Witness:** Dylan Vallen	
Criteria for Success: ☒ Passes – If each displayed pass/fail indictor matches the plotted data metric (Lane Keep, Lateral Acceleration, Longitude Acceleration, Curvature, Curvature Rate, Jerk, and Collision Detection) relative to the threshold ☐ Fails – If each displayed pass/fail indictor does not match the plotted data metric (Lane Keep, Lateral Acceleration, Longitude Acceleration, Curvature, Curvature Rate, Jerk, and Collision Detection) relative to the threshold	
Test Procedure: 1. Access the AutoDrive-RoadRunner GitHub repository and download the files 2. Run MATLAB 2024b and open AutoDrive-RoadRunner\RR_GUI.mlapp 3. Run RR_GUI.mlapp and Click – Choose Installation Path 4. Select your RoadRunner Installation Path through your file explorer, the file should end with \RoadRunner R2024b\bin\win64 5. Click – Choose Project Path 6. Select your RoadRunner Project Path through your file explorer – the needed file is AutoDrive-RoadRunner 7. Select a scene – Four Way Stop, an obstacle – Clear, and a behavior – Straight 8. Click – Run Simulation 9. After the run completes the output GUI will populate, review the displayed pass/fail indicators and compare the indicator to the corresponding data metric plot behavior	

10. Confirm whether the plotted data stays within or exceeds the threshold 11. Verify that each indicator matches each of the corresponding actual data metric behavior	
Test Results: See Appendix B – Testing Results: 4.2. Output UI Pass/Fail Indicator Test Validation Screenshot (1-7)	
Action Items: N/A	
Test Engineer: Walker Sheidow, 3/31/2026	Witness: Dylan Vallen, 3/31/2026